

Reviewer Pack — Selberg Log-Gas Rigidity Defect Bounds SOS-2 Max-CUT Gap

Entry 14fdafc31b7e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 11:07:39 UTC

Selberg Log-Gas Rigidity Defect Bounds SOS-2 Max-CUT Gap

Entry ID: 14fdafc31b7e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 11:07:39 UTC

1. Conjecture

Field A (mathematical branch): Random matrix theory: the $\beta=2$ Selberg-integral / log-gas free energy $F[\lambda] = \sum_{\{i<j\}} \log|\lambda_i - \lambda_j|$ measuring spectral rigidity relative to the Wigner-semicircle / Kesten-McKay quantile law (Mehta, Forrester, Wigner-Dyson ‘rigidity’ tradition), used in β -ensemble fluctuation theory but essentially absent from SOS / CSP integrality-gap literature and provably distinct from free-probability cumulant calculus **Field B** (complexity object): Sum-of-squares degree-2 (Lasserre level-2 / Goemans-Williamson basic SDP) integrality gap for unweighted max-CUT on 3-regular graphs, the canonical SOS hierarchy CSP target

Statement:

For a 3-regular graph G on n vertices, let $\lambda_1 \geq \dots \geq \lambda_n$ be the eigenvalues of A_G and let $\mu_i := 2\sqrt{2} \cdot \cos((i-1/2)\pi/n)$ be the Kesten-McKay quantile points (the $d=3$ Wigner-semicircle inverse-CDF on n equispaced probabilities). Define the Selberg log-defect $SLD(G) := (2/n^2) \cdot [\sum_{\{i<j\}} \log|\lambda_i - \lambda_j| - \sum_{\{i<j\}} \log|\mu_i - \mu_j|]$, an unsigned spectral-rigidity score where $SLD(G) < 0$ means strictly more rigid than the semicircle. Conjecture: there exists an absolute constant $c \geq 1/8$ such that for every 3-regular graph G , $SDP_2(G)/MC(G) \geq 1 + c \cdot \max(0, -SLD(G))^2$, where $SDP_2(G)$ is the GW basic-SDP value $\max_{\{X \sqsupseteq 0, \text{diag}(X)=1\}} (|E| - \langle A_G, X \rangle / 2) / 2$ and $MC(G)$ is the exact max-cut; equivalently any 0.879 -approximating moment matrix has $SLD(G) \geq -\sqrt{(0.121/c)}$.

Rationale (proposer’s reasoning):

Random 3-regular graphs are Ramanujan-like and $SLD(G) \approx 0$, the regime where SDP is tightest; off-spectrum graphs with crystalline (rigid, sub-Wigner) bulk are exactly those whose few outlier eigenvalues drive a slack $\langle A_G, X \rangle$ in the GW relaxation. Log-gas rigidity is a coordinate-free, S_n -natural invariant — orthogonal in technique to Fourier/hypercontractivity, kurtosis, and Positivstellensatz arguments that have already failed on this target — so the bridge probes new structure: rigidity-induced over-relaxation. The mapping is GCT-flavored in that the $\beta=2$ Selberg integral is the partition function of the $U(n)$ -invariant Haar measure on Hermitian conjugacy classes, the same representation-theoretic object Mulmuley invokes in plethystic asymptotics.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aa56ae9c9e7f0cfa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 random 3-regular graphs ($n \in \{8, 10, 12, 14, 16\}$, 30 each), the conjecture is SUPPORTED iff Spearman ρ between $\max(0, -\text{SLD}(G))^2$ and $\text{SDP}_2(G)/\text{MC}(G) - 1$ is ≥ 0.40 AND no instance has $\text{SLD}(G) \leq -0.05$ together with $\text{SDP}_2(G)/\text{MC}(G) \leq 1.002$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Selberg log-gas spectral rigidity max-cut SDP integrality gap - Kesten-McKay eigenvalue rigidity 3-regular Goemans-Williamson SDP - log-determinant beta-ensemble Lasserre sum-of-squares cubic graph max-cut

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    n = 10 # Default value for n, will be overridden in the loop
    degrees = [3] * n # Default degree for each vertex, will be overridden in the loop
    adjacency_matrix = [[0] * n for _ in range(n)]

    def generate_graph():
        nonlocal n, degrees, adjacency_matrix
        while True:
            n = random.choice([8, 10, 12, 14, 16])
            degrees = [3] * n
            adjacency_matrix = [[0] * n for _ in range(n)]

            edges = set()
            for i in range(n):
                neighbors = random.sample(range(n), degrees[i])
                for j in neighbors:
                    if i < j and (i, j) not in edges and (j, i) not in edges:
```

```

        adjacency_matrix[i][j] = 1
        adjacency_matrix[j][i] = 1
        edges.add((i, j))

    # Check if the graph is 3-regular
    if all(sum(row) == degrees[i] for i, row in enumerate(adjacency_matrix)):
        break

def eigenvalues(matrix):
    n = len(matrix)
    eigenvals = []
    for _ in range(20): # Power iteration method to approximate eigenvalues
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x * x for x in v))
        Av = [sum(matrix[i][j] * v[j] for j in range(n)) for i in range(n)]
        lambda_ = sum(Av[i] * v[i] for i in range(n)) / sum(v[i] * v[i] for i in range(n))
        eigenvals.append(lambda_)
    return sorted(eigenvals)

def sld(g, mu):
    n = len(g)
    log_defect = 0
    for i, j in combinations(range(n), 2):
        log_defect += math.log(abs(g[i][j] - g[j][i])) - math.log(abs(mu[i] - mu[j]))
    return (2 / n**2) * log_defect

def sdp_2(g):
    n = len(g)
    diag = [1] * n
    for _ in range(200): # Projected gradient method to approximate SDP_2
        X = [[diag[i] if i == j else 0 for j in range(n)] for i in range(n)]
        grad = [[0] * n for _ in range(n)]
        for i, j in combinations(range(n), 2):
            grad[i][j] = (g[i][j] - X[i][j]) / 2
            grad[j][i] = (g[i][j] - X[j][i]) / 2
        diag_sum = sum(diag)
        for i in range(n):
            diag[i] -= 0.1 * (diag_sum - n)
        for i, j in combinations(range(n), 2):
            X[i][j] -= 0.1 * grad[i][j]
            X[j][i] -= 0.1 * grad[j][i]
    return sum(X[i][j] for i, j in combinations(range(n), 2)) / n**2

def mc(g):
    n = len(g)
    max_cut_value = -1
    for mask in range(1 << n):
        cut_size = bin(mask).count('1')
        if cut_size > n // 2:
            continue
        cut_value = sum(sum(g[i][j] * (mask & (1 << i)) and (mask & (1 << j)) == 0 for j in range(n))
        max_cut_value = max(max_cut_value, cut_value)
    return max_cut_value

generate_graph()
lambda_ = eigenvalues(adjacency_matrix)

```

```

mu = [2 * math.sqrt(2) * math.cos((i - 0.5) * math.pi / n) for i in range(n)]
sld_g = sld(lambda_, mu)
mc_g = mc(adjacency_matrix)
sdp_2_g = sdp_2(adjacency_matrix)

return {
    "metric_name": "SDP_2/MC - SLD(G)^2",
    "metric_value": sdp_2_g / mc_g - 1,
    "instances_tested": 1,
    "conjecture_holds": True if sld_g >= -math.sqrt(0.121 / (1/8)) else False,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [3, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        random.seed(seed)
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"SLD(G) < -0.05 with SDP_2(G)/MC(G) <= 1.002\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_832d5e7a.py", line 109, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_832d5e7a.py", line 89, in run_trial
    lambda_ = eigenvalues(adjacency_matrix)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_832d5e7a.py", line 47, in eigenvalues
    v /= math.sqrt(sum(x * x for x in v))
TypeError: unsupported operand type(s) for /=: 'list' and 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `eigenvalues` routine (`list != float`), so no data was produced and the pre-registered Spearman/instance criteria could not be evaluated.
| next: Fix the eigenvector normalization bug (use a numpy array or divide elementwise) and rerun the 150-instance trial.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	276219
2	preregistration	claude_max	opus	0	0	5313
3	novelty	claude_max	opus	0	0	3777
4	novelty	claude_max	opus	0	0	5625
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19348
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16018
7	judge	claude_max	opus	0	0	12960

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 339260 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/14fdafc31b7e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/14fdafc31b7e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/14fdafc31b7e.tar.gz` (if generated)